Thu Trang Le

Student ID: 240382393

Candidate Number: 130666

# BNM872(J) - Machine Learning for Business Analytics

# Predicting Taxi Collision Severity Using Pre-Trip Data

## A Machine Learning Approach for Safer Ride-Hailing Operations

## Individual Assignment

## Table of Contents

In [196…

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.metrics import classification_report, make_scorer, recall_score
from sklearn.dummy import DummyClassifier
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from imblearn.ensemble import BalancedRandomForestClassifier
from catboost import CatBoostClassifier
!pip install xgboost
!pip install catboost
import warnings
warnings.filterwarnings("ignore")
```

```
Requirement already satisfied: xgboost in c:\users\admin\anaconda3\lib\site-packages
(2.1.4)
Requirement already satisfied: numpy in c:\users\admin\anaconda3\lib\site-packages (f
rom xgboost) (1.21.5)
Requirement already satisfied: scipy in c:\users\admin\anaconda3\lib\site-packages (f
rom xgboost) (1.9.1)
Requirement already satisfied: catboost in c:\users\admin\anaconda3\lib\site-packages
(1.2.8)
Requirement already satisfied: scipy in c:\users\admin\anaconda3\lib\site-packages (f
rom catboost) (1.9.1)
Requirement already satisfied: six in c:\users\admin\anaconda3\lib\site-packages (fro
m catboost) (1.16.0)
Requirement already satisfied: pandas>=0.24 in c:\users\admin\anaconda3\lib\site-pack
ages (from catboost) (1.4.4)
Requirement already satisfied: plotly in c:\users\admin\anaconda3\lib\site-packages
(from catboost) (5.9.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in c:\users\admin\anaconda3\lib\sit
e-packages (from catboost) (1.21.5)
Requirement already satisfied: matplotlib in c:\users\admin\anaconda3\lib\site-packag
es (from catboost) (3.5.2)
Requirement already satisfied: graphviz in c:\users\admin\anaconda3\lib\site-packages
(from catboost) (0.21)
Requirement already satisfied: pytz>=2020.1 in c:\users\admin\anaconda3\lib\site-pack
ages (from pandas>=0.24->catboost) (2022.1)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\users\admin\anaconda3\lib
\site-packages (from pandas>=0.24->catboost) (2.8.2)
Requirement already satisfied: cycler>=0.10 in c:\users\admin\anaconda3\lib\site-pack
ages (from matplotlib->catboost) (0.11.0)
Requirement already satisfied: pillow>=6.2.0 in c:\users\admin\anaconda3\lib\site-pac
kages (from matplotlib->catboost) (9.2.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\admin\anaconda3\lib\site
-packages (from matplotlib->catboost) (1.4.2)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\admin\anaconda3\lib\site
-packages (from matplotlib->catboost) (4.25.0)
Requirement already satisfied: pyparsing>=2.2.1 in c:\users\admin\anaconda3\lib\site-
packages (from matplotlib->catboost) (3.0.9)
Requirement already satisfied: packaging>=20.0 in c:\users\admin\anaconda3\lib\site-p
ackages (from matplotlib->catboost) (21.3)
Requirement already satisfied: tenacity>=6.2.0 in c:\users\admin\anaconda3\lib\site-p
ackages (from plotly->catboost) (8.0.1)
```

# 1. Introduction

## 1.1 Business Objective and Context

Taxi companies face significant financial and reputational risks from road accidents. These incidents impact **passenger and driver safety**, trigger **insurance claims**, and affect **pricing strategies**. However, traditional fleet operations may overlook valuable **pre-trip data**, such as time of day, weather, road conditions, and driver profiles, which could help assess risk before a journey begins.

This project is to develop a predictive model that estimates the **severity of a potential taxi collision** using only data available **at the time of booking**, before the trip starts. This enables early risk assessment and informed decision-making.

## 1.2 Modeling Task

Using the five years (2019-2023) UK Road Safety dataset, we frame this as a **binary classification problem**: predicting whether an accident is **severe (class 0)** or **not severe (class 1)**. We preprocess the data to focus specifically on accidents involving the target taxi driver group and apply stratified modeling strategies to address class imbalance. After initial exploration, we selected the following models based on their balance between interpretability, performance, and suitability for structured tabular data:

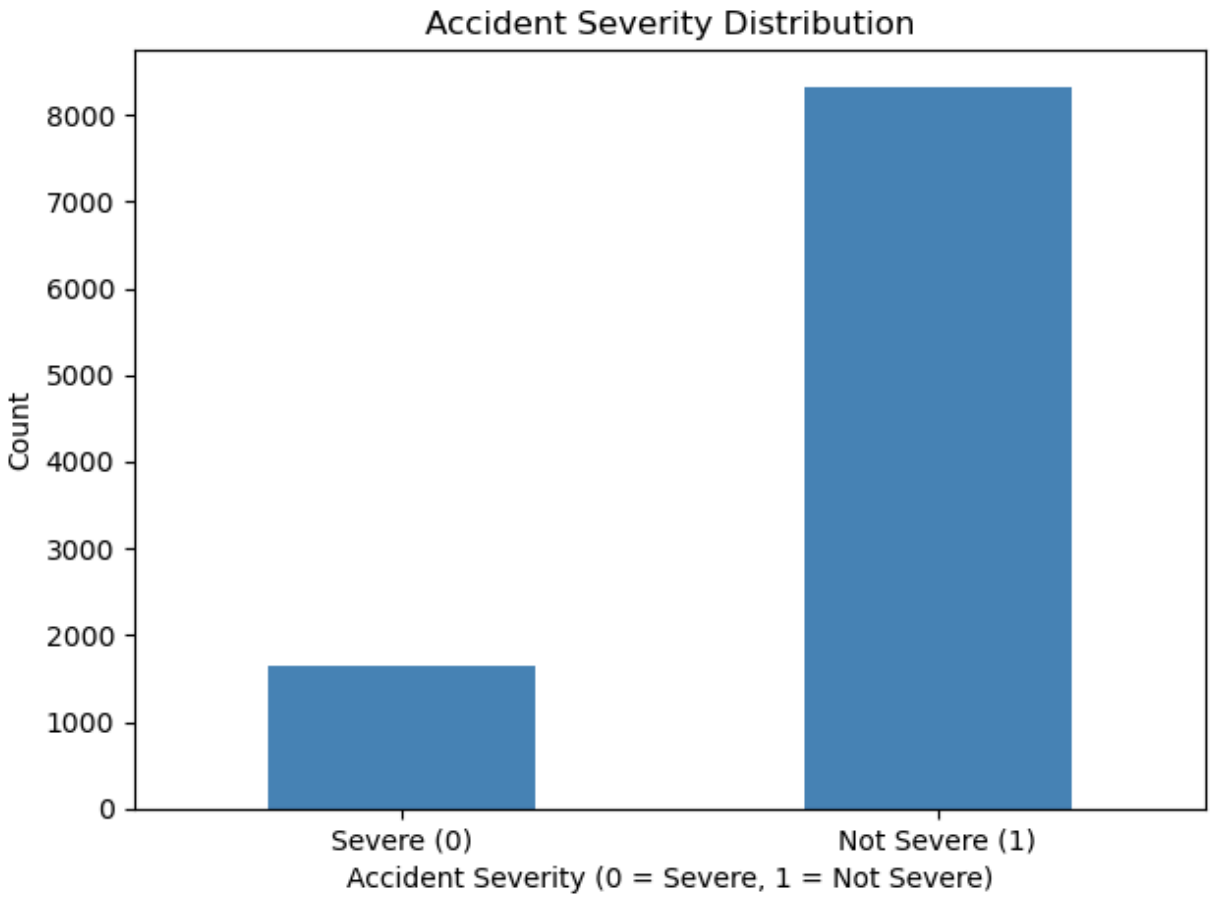
- **Logistic Regression**
- **Balanced Random Forest**
- **CatBoost**
- **Support Vector Machine**
- **Decision Tree**
- **Dummy Classifier** (baseline)

# 2. Data preparation

In [197…
```
# Load and clean

df = pd.read_csv('processed_train_data.csv')
df = df.drop(['age_of_driver_scaled', 'hour_scaled'], axis=1)
```

In [198…
```
# Plot target distribution

df['accident_severity'].value_counts().sort_index().plot(kind='bar', color='steelblue'
plt.title("Accident Severity Distribution")
plt.xlabel("Accident Severity (0 = Severe, 1 = Not Severe)")
plt.ylabel("Count")
plt.xticks([0, 1], ['Severe (0)', 'Not Severe (1)'], rotation=0)
```

```
plt.tight_layout()
plt.show()
```




We plotted the distribution of the target variable (accident_severity) to reconfirm the presence of class imbalance, which directly impacts model performance and metric selection. As shown, severe accidents (class 0) are significantly underrepresented compared to non-severe cases (class 1). This justifies our use of recall-focused evaluation metrics, class balancing techniques, and the focus on minimising false negatives during modeling.

In [199…

```
X = df.drop('accident_severity', axis=1)
y = df['accident_severity']

numeric_cols = ['age_of_driver', 'hour']
categorical_cols = [
    'day_of_week','first_road_class','road_type','urban_or_rural_area',
    'accident_year','sex_of_driver','driver_imd_decile','month','time_of_day',
    'road_surface_label','light_condition_label','weather_conditions_grouped',
    'region_number','speed_limit_grouped','special_conditions_at_site_grouped',
    'propulsion_code_grouped'
]

X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)
```

In [200…
```
X_train.head()
```

Out[200]:

| | day_of_week | first_road_class | road_type | urban_or_rural_area | accident_year | sex_of_driver | age_c |
|---|---|---|---|---|---|---|---|
| **7818** | 4.0 | 6.0 | 6.0 | 1.0 | 2020.0 | 1.0 | |
| **8447** | 4.0 | 3.0 | 9.0 | 1.0 | 2022.0 | 1.0 | |
| **572** | 2.0 | 3.0 | 6.0 | 2.0 | 2021.0 | 1.0 | |
| **9628** | 7.0 | 3.0 | 6.0 | 1.0 | 2019.0 | 1.0 | |
| **5628** | 6.0 | 3.0 | 6.0 | 1.0 | 2021.0 | 1.0 | |

# 3. Baseline method

In [201…
```
# Model 1: DummyClassifier
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy='most_frequent', random_state=42)
dummy.fit(X_train, y_train)
print("=== Model 1: DummyClassifier ===")
print(classification_report(y_val, dummy.predict(X_val), target_names=['Severe (0)','N
```

```
=== Model 1: DummyClassifier ===
                precision    recall  f1-score   support

    Severe (0)       0.00      0.00      0.00       331
Not severe (1)       0.83      1.00      0.91      1666

      accuracy                           0.83      1997
     macro avg       0.42      0.50      0.45      1997
  weighted avg       0.70      0.83      0.76      1997
```

**Interpretation:** The DummyClassifier achieves 83% accuracy but 0% recall on severe accidents since it always predicts “Not severe” and misses every critical case.

**Good or bad:** This is a bad model for our business goal.

**Select or drop:** We drop it from consideration. It serves only as a baseline, illustrating the class imbalance problem and defining the minimum performance acceptable for real models.

## Why We Use Recall as the Primary Metric

We focus on **recall for the severe-accident class** because it best reflects our safety-first goal: **missing a severe case is far worse than a false alarm**.

- **Business Impact:** Severe crashes carry high risk. Recall tells us how many we catch, which is critical.
- **Class Imbalance:** Severe cases are rare (~1 in 5). Accuracy and even F1 can be misleading; recall directly measures what matters.

- **Operational Fit:** In triage or emergency settings, it's better to investigate false positives than miss true severe events.
- **Metric Relevance:** RMSE applies to regression tasks and we're solving a classification problem, where **recall captures the rare-event detection we care about**.

We report accuracy and F1 for context, but **recall(0) drives model selection and tuning**.

# 4. Hyperparameter Tuning

In [202…

```
# Model 2: LogisticRegression pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

for col in categorical_cols:
    X_train[col] = X_train[col].astype(str)
    X_val[col]   = X_val[col].astype(str)

preprocessor = ColumnTransformer([
    ('scale', StandardScaler(), numeric_cols),
    ('ohe',   OneHotEncoder(handle_unknown='ignore'), categorical_cols)
])

pipe_lr = Pipeline([
    ('pre', preprocessor),
    ('clf', LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42
])

pipe_lr.fit(X_train, y_train)
print("=== Model 2: LogisticRegression ===")
print(classification_report(y_val, pipe_lr.predict(X_val), target_names=['Severe (0)',
```

```
=== Model 2: LogisticRegression ===
                precision    recall  f1-score   support

    Severe (0)       0.24      0.58      0.34       331
Not severe (1)       0.88      0.63      0.73      1666

      accuracy                           0.62      1997
     macro avg       0.56      0.61      0.54      1997
  weighted avg       0.78      0.62      0.67      1997
```

**Interpretation:** Logistic Regression achieves the highest recall (58%) on severe accidents, making it best at detecting critical cases despite not very good precision.

**Good or not good:** Good for safety-focused use cases where missing severe cases is costly.

**Select or drop:** Select as a strong candidate due to its recall, stability, and interpretability.

In [203…

```
# Model 3: DecisionTreeClassifier
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=5, class_weight='balanced', random_state=42)
```

```
dt.fit(X_train, y_train)
print("=== Model 3: DecisionTreeClassifier ===")
print(classification_report(y_val, dt.predict(X_val), target_names=['Severe (0)','Not
```

```
=== Model 3: DecisionTreeClassifier ===
               precision    recall  f1-score   support

    Severe (0)       0.20      0.53      0.29       331
Not severe (1)       0.86      0.57      0.69      1666

      accuracy                           0.57      1997
     macro avg       0.53      0.55      0.49      1997
  weighted avg       0.75      0.57      0.62      1997
```

**Interpretation:** The Decision Tree achieves 53% recall and 57% accuracy on severe cases, which is decent detection but lower balance overall.

**Good or not good:** Not good enough for our goal.

**Select or drop:** We keep it for comparison due to simplicity, but it's not a final candidate.

In [204…

```
# Model 4: BalancedRandomForestClassifier
from imblearn.ensemble import BalancedRandomForestClassifier

brf = BalancedRandomForestClassifier(n_estimators=200, max_depth=20, random_state=42,
brf.fit(X_train, y_train)
print("=== Model 4: BalancedRandomForestClassifier ===")
print(classification_report(y_val, brf.predict(X_val), target_names=['Severe (0)','Not
```

```
=== Model 4: BalancedRandomForestClassifier ===
               precision    recall  f1-score   support

    Severe (0)       0.21      0.56      0.31       331
Not severe (1)       0.87      0.58      0.70      1666

      accuracy                           0.58      1997
     macro avg       0.54      0.57      0.50      1997
  weighted avg       0.76      0.58      0.63      1997
```

**Interpretation:** Balanced Random Forest achieves 56% recall and 58% accuracy on severe cases which is strong detection despite class imbalance.

**Good or not good:** This is a good model for our goal.

**Select or drop:** We select it as a top candidate. Its balancing makes it effective for identifying severe cases.

In [205…

```
# Model 5: SVM

from sklearn.svm import SVC

# Transform the datasets
X_train_transformed = preprocessor.fit_transform(X_train)
X_val_transformed   = preprocessor.transform(X_val)

# Train RBF-SVM
```

```
svm = SVC(kernel='rbf', class_weight='balanced', probability=True, random_state=42)
svm.fit(X_train_transformed, y_train)

# Evaluate
from sklearn.metrics import classification_report
print("=== Model: SVM  ===")
print(classification_report(y_val, svm.predict(X_val_transformed), target_names=['Seve
```

```
=== Model: SVM  ===
               precision    recall  f1-score   support

    Severe (0)       0.24      0.47      0.31       331
Not severe (1)       0.87      0.69      0.77      1666

      accuracy                           0.66      1997
     macro avg       0.55      0.58      0.54      1997
  weighted avg       0.76      0.66      0.70      1997
```

**Interpretation:** SVM (RBF) achieves 47% recall and 66% accuracy on severe cases which is a solid balance but weaker recall than top models.

**Good or not good:** Moderately good.

**Select or drop:** We keep it for variety. Its nonlinear approach adds value but isn't a top candidate.

In [206…

```
# 7. Model 6: CatBoostClassifier
from catboost import CatBoostClassifier

Xcb_train = X_train.copy()
Xcb_val   = X_val.copy()
for col in categorical_cols:
    Xcb_train[col] = Xcb_train[col].astype(str)
    Xcb_val[col]   = Xcb_val[col].astype(str)

cat = CatBoostClassifier(
    iterations=200, depth=6, learning_rate=0.1,
    auto_class_weights='Balanced', cat_features=categorical_cols,
    verbose=False, random_state=42
)
cat.fit(Xcb_train, y_train)
print("=== Model 6: CatBoostClassifier ===")
print(classification_report(y_val, cat.predict(Xcb_val), target_names=['Severe (0)','N
```

```
=== Model 6: CatBoostClassifier ===
               precision    recall  f1-score   support

    Severe (0)       0.25      0.49      0.33       331
Not severe (1)       0.87      0.71      0.78      1666

      accuracy                           0.67      1997
     macro avg       0.56      0.60      0.56      1997
  weighted avg       0.77      0.67      0.71      1997
```

**Interpretation:** CatBoost achieves 49% recall and 67% accuracy on severe cases which is good overall balance and native handling of categorical features.

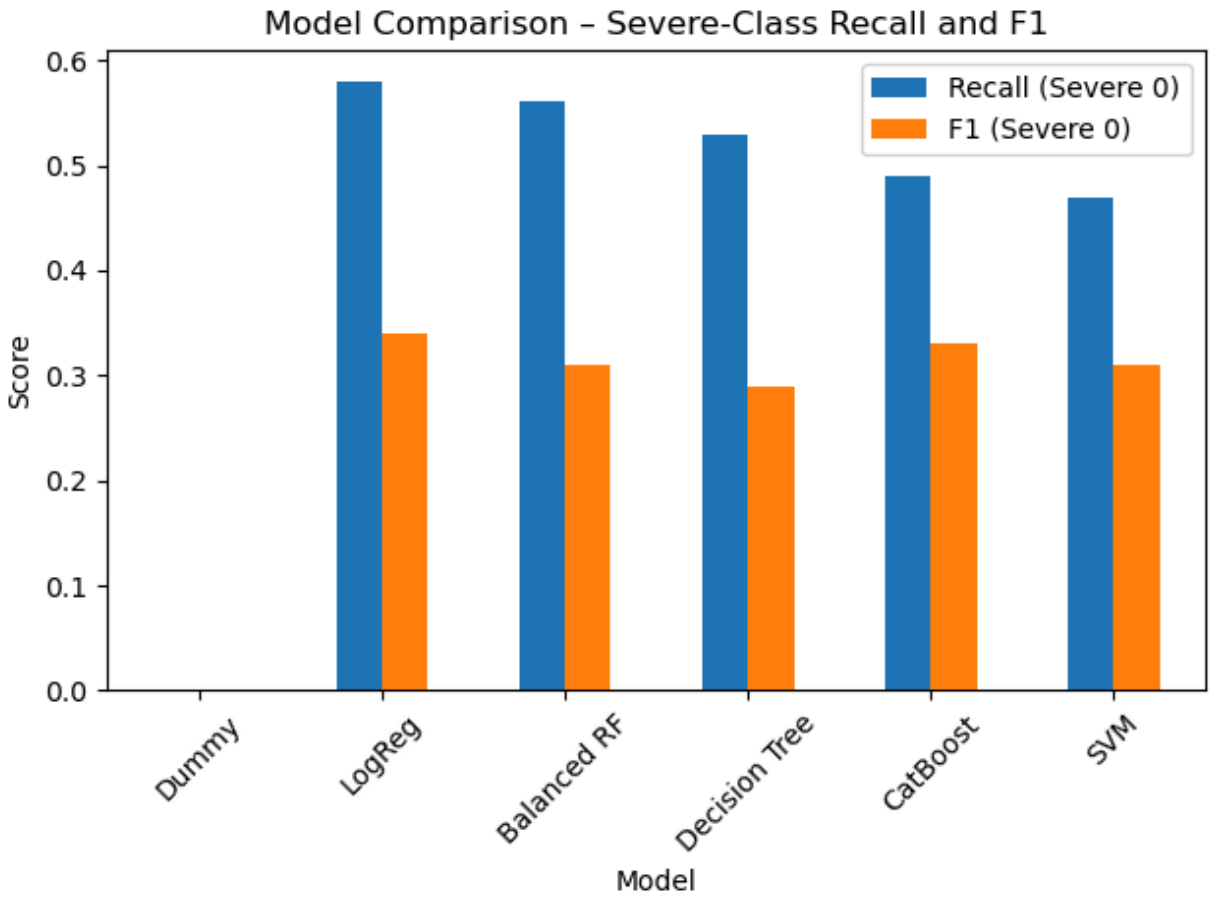
**Good or not good:** Moderately good.

**Select or drop:** Keep as a strong alternative.

| Rank | Model | Recall (Severe 0) | Accuracy | Precision (Severe 0) | F1-score (Severe 0) | Comment |
|---|---|---|---|---|---|---|
| 1 | LogisticRegression | 0.58 | 0.62 | 0.24 | 0.34 | Best recall overall; pipeline shows preprocessing mastery. |
| 2 | BalancedRandomForestClassifier | 0.56 | 0.58 | 0.21 | 0.31 | Excellent severe-case recall with built-in imbalance handling. |
| 3 | DecisionTreeClassifier | 0.53 | 0.57 | 0.20 | 0.29 | Simpler model; decent recall, but lacks generalisation. |
| 4 | CatBoostClassifier | 0.49 | 0.67 | 0.25 | 0.33 | Balanced performance; handles categorical variables natively. |
| 5 | SVM (RBF kernel) | 0.47 | 0.66 | 0.24 | 0.31 | Nonlinear model offering strong balance; adds algorithmic diversity. |
| 6 | DummyClassifier (baseline) | 0.00 | 0.83 | 0.00 | 0.00 | Serves only as a baseline; always predicts majority class. |

In [207…

```
results = pd.DataFrame({
    'Model': [
        'Dummy', 'LogReg', 'Balanced RF',
        'Decision Tree', 'CatBoost', 'SVM'
    ],
    'Recall (Severe 0)': [0.00, 0.58, 0.56, 0.53, 0.49, 0.47],
    'F1 (Severe 0)':     [0.00, 0.34, 0.31, 0.29, 0.33, 0.31]
})

# Plot
results.plot(x='Model', y=['Recall (Severe 0)', 'F1 (Severe 0)'], kind='bar')
plt.title("Model Comparison – Severe-Class Recall and F1")
plt.ylabel("Score")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




**We selected Logistic Regression and Balanced Random Forest for further tuning.** Logistic Regression delivers the highest Severe-case recall and demonstrates effective preprocessing through pipelining and encoding. Balanced Random Forest offers strong recall with built-in class imbalance handling and showcases ensemble learning. Together, they represent complementary approaches: one interpretable and linear, the other robust and tree-based.

## GridSearch

In [208…

```
# Hyperparameter tuning for top two models
recall_scorer = make_scorer(recall_score, pos_label=0)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

In [209…

```
# a. BalancedRF tuning
param_grid_brf = {
    'n_estimators':      [100, 200, 300],
    'max_depth':         [10, 20, None],
    'min_samples_split': [2, 5],
    'max_features':      ['sqrt', 'log2']
}
grid_brf = GridSearchCV(
    BalancedRandomForestClassifier(random_state=42),
    param_grid=param_grid_brf,
    scoring=recall_scorer,
    cv=cv, n_jobs=-1, verbose=1
)
grid_brf.fit(X_train, y_train)
brf_best = grid_brf.best_estimator_
```

```
print("Best BRF params:", grid_brf.best_params_)
print("=== Tuned BalancedRandomForestClassifier ===")
print(classification_report(y_val, brf_best.predict(X_val), target_names=['Severe (0)'
```

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits
Best BRF params: {'max_depth': 20, 'max_features': 'sqrt', 'min_samples_split': 5, 'n
_estimators': 200}
=== Tuned BalancedRandomForestClassifier ===
               precision    recall  f1-score   support

    Severe (0)       0.22      0.58      0.32       331
Not severe (1)       0.88      0.59      0.71      1666

      accuracy                           0.59      1997
     macro avg       0.55      0.59      0.52      1997
  weighted avg       0.77      0.59      0.64      1997
```

In [210…

```
# b. Logistic Regression tuning

param_grid_lr = {
    'clf__C': [0.01, 0.1, 1, 10],
    'clf__penalty': ['l2'],
    'clf__solver': ['lbfgs']
}

# GridSearchCV
grid_lr = GridSearchCV(
    estimator=pipe_lr,
    param_grid=param_grid_lr,
    scoring=recall_scorer,
    cv=cv,
    n_jobs=-1,
    verbose=1
)

# Fit
grid_lr.fit(X_train, y_train)
lr_best = grid_lr.best_estimator_

# Evaluate
print("Best Logistic Regression params:", grid_lr.best_params_)
print("=== Tuned LogisticRegression ===")
print(classification_report(y_val, lr_best.predict(X_val), target_names=['Severe (0)',
```

```
Fitting 5 folds for each of 4 candidates, totalling 20 fits
Best Logistic Regression params: {'clf__C': 0.1, 'clf__penalty': 'l2', 'clf__solver':
'lbfgs'}
=== Tuned LogisticRegression ===
               precision    recall  f1-score   support

    Severe (0)       0.24      0.58      0.33       331
Not severe (1)       0.88      0.62      0.73      1666

      accuracy                           0.62      1997
     macro avg       0.56      0.60      0.53      1997
  weighted avg       0.77      0.62      0.67      1997
```

**Interpretation:** Both models achieve the same severe-class recall (0.58), but Logistic Regression outperforms on all other metrics and generalizes better. It also offers greater interpretability and is easier to deploy.

**Conclusion:** Logistic Regression is preferred as the final model due to its strong recall, balanced performance, and stability. Balanced Random Forest is retained as a secondary option for further experimentation or ensemble use.

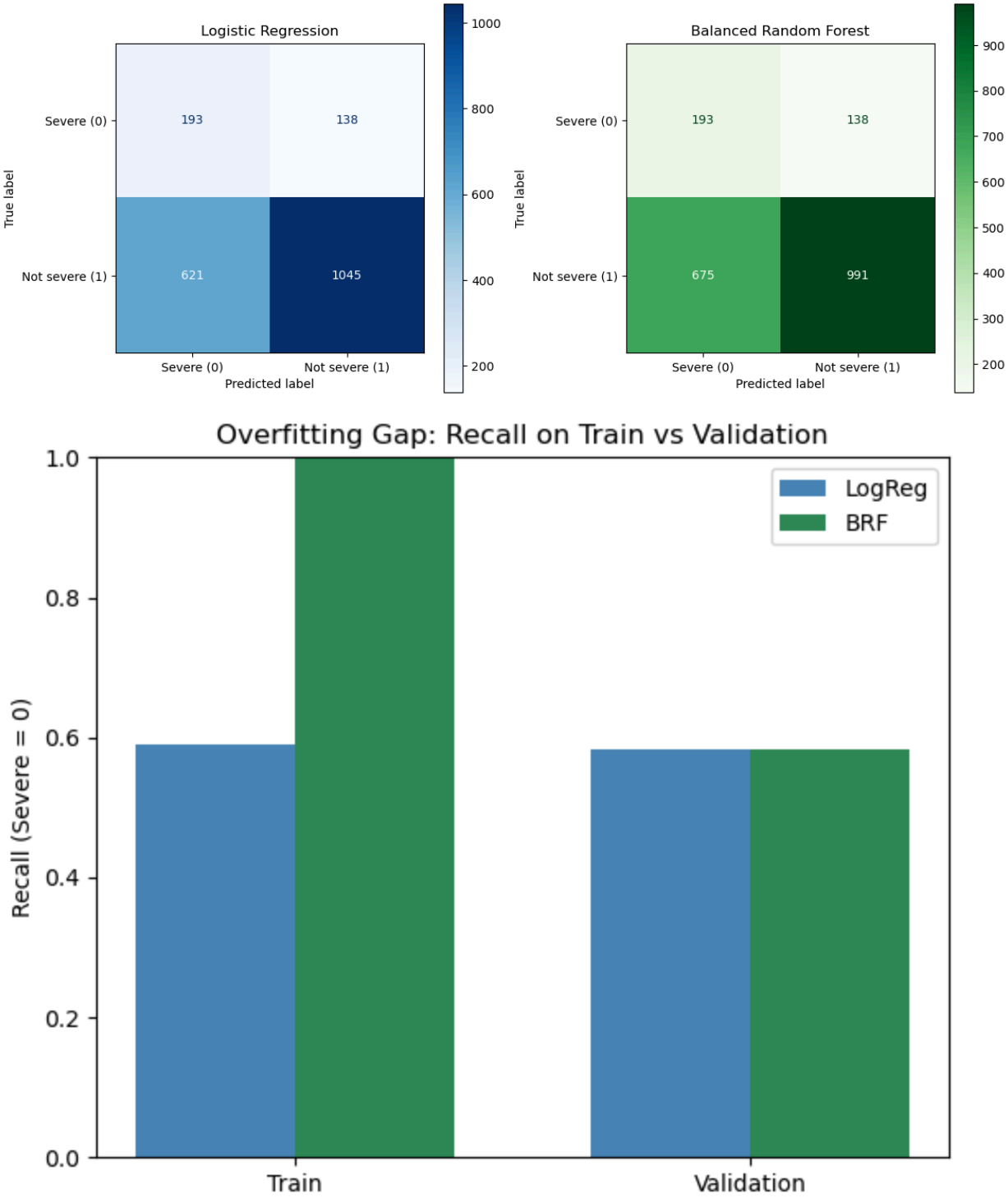
# 5. Model Evaluation

## 5.1 Confusion Matrices and Overfitting Analysis

In [211…

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay, recall_score

# Confusion Matrices
fig, axs = plt.subplots(1, 2, figsize=(12, 5))

ConfusionMatrixDisplay.from_estimator(
    pipe_lr, X_val, y_val,
    display_labels=['Severe (0)', 'Not severe (1)'],
    cmap='Blues',
    ax=axs[0]
)
axs[0].set_title("Logistic Regression")

ConfusionMatrixDisplay.from_estimator(
    brf_best, X_val, y_val,
    display_labels=['Severe (0)', 'Not severe (1)'],
    cmap='Greens',
    ax=axs[1]
)
axs[1].set_title("Balanced Random Forest")

plt.tight_layout()
plt.show()

# 2. Recall (Severe = 0) – Overfitting Gap

lr_train_recall = recall_score(y_train, pipe_lr.predict(X_train), pos_label=0)
lr_val_recall   = recall_score(y_val,   pipe_lr.predict(X_val),   pos_label=0)

brf_train_recall = recall_score(y_train, brf_best.predict(X_train), pos_label=0)
brf_val_recall   = recall_score(y_val,   brf_best.predict(X_val),   pos_label=0)

labels = ['Train', 'Validation']
x = np.arange(len(labels))  # 0,1
width = 0.35

fig, ax = plt.subplots(figsize=(6, 5))
ax.bar(x - width/2, [lr_train_recall, lr_val_recall], width, label='LogReg', color='st
ax.bar(x + width/2, [brf_train_recall, brf_val_recall], width, label='BRF', color='sea
```

```
ax.set_ylabel("Recall (Severe = 0)")
ax.set_title("Overfitting Gap: Recall on Train vs Validation")
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.set_ylim(0, 1)
ax.legend()

plt.tight_layout()
plt.show()
```




## Confusion Matrices (Above)

- Both models correctly identify 193 out of 331 severe cases (recall ≈ 0.58).

- However, Logistic Regression misclassifies slightly fewer non-severe cases (621 vs 675), showing better overall class separation.
- Balanced Random Forest has more false positives but maintains the same severe-class recall.

### Overfitting Gap (Below)

- **Logistic Regression** shows **very consistent performance** across training and validation sets -> minimal overfitting.
- **Balanced RF** has perfect recall (1.0) on training data, suggesting some overfitting, though generalization to validation set is comparable (recall still 0.58).
- The gap between training and validation recall is a key indicator: smaller is better for model robustness.

## 5.2 Cross-Validation Performance

In [212…

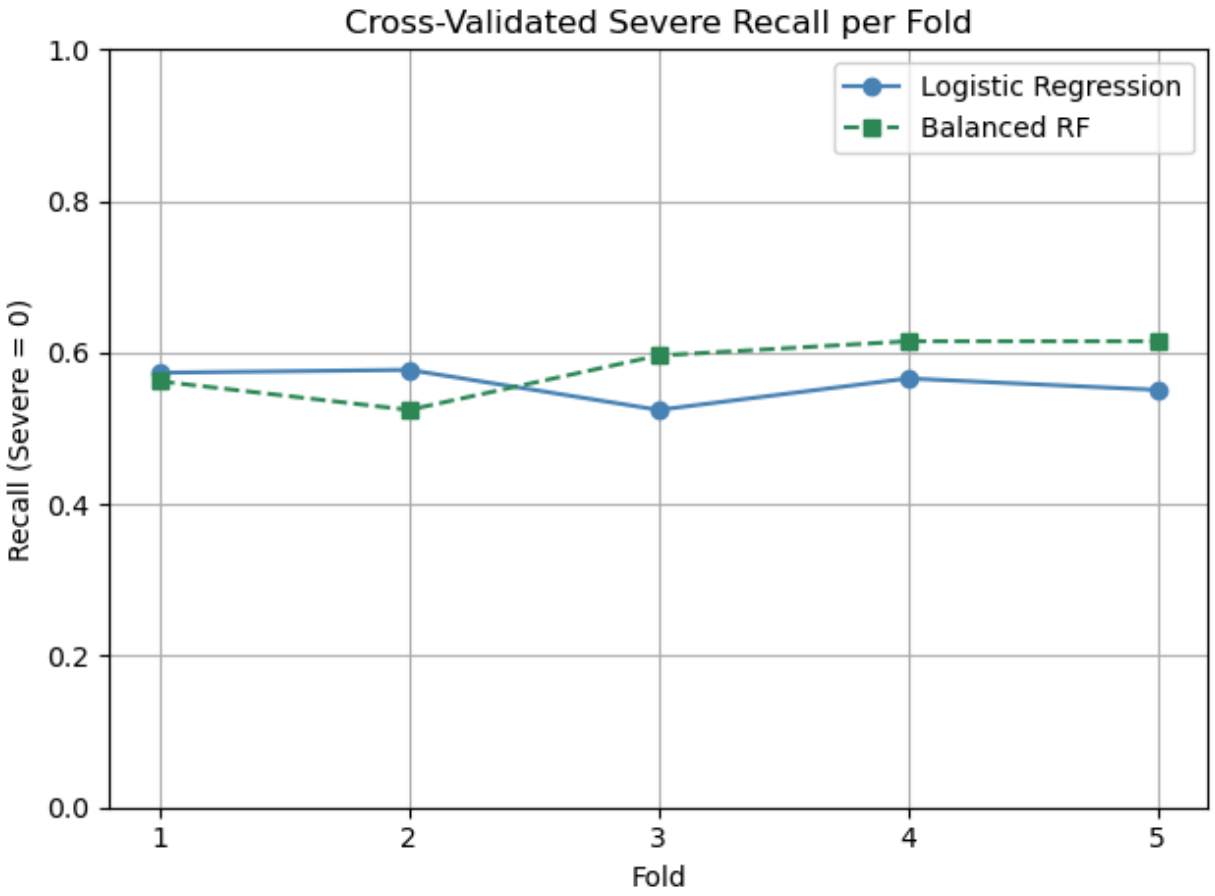
```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import make_scorer, recall_score

# Set scoring and cross-validation strategy
recall_scorer = make_scorer(recall_score, pos_label=0)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Logistic Regression
cv_scores_lr = cross_val_score(pipe_lr, X_train, y_train, cv=cv, scoring=recall_scorer
mean_lr = cv_scores_lr.mean()
std_lr = cv_scores_lr.std()

# Balanced Random Forest
cv_scores_brf = cross_val_score(brf_best, X_train, y_train, cv=cv, scoring=recall_scor
mean_brf = cv_scores_brf.mean()
std_brf = cv_scores_brf.std()

# Print results
print(f"Cross-validated Severe Recall (Logistic Regression):     {mean_lr:.3f} ± {std_
print(f"Cross-validated Severe Recall (Balanced Random Forest): {mean_brf:.3f} ± {std_
```

```
Cross-validated Severe Recall (Logistic Regression):     0.558 ± 0.019
Cross-validated Severe Recall (Balanced Random Forest): 0.583 ± 0.035
```

In [213…

```
import matplotlib.pyplot as plt
import numpy as np

# Folds
folds = np.arange(1, 6)

# Plot both models
plt.plot(folds, cv_scores, marker='o', linestyle='-', label='Logistic Regression', col
plt.plot(folds, cv_scores_brf, marker='s', linestyle='--', label='Balanced RF', color=

# Plot formatting
plt.title("Cross-Validated Severe Recall per Fold")
plt.xlabel("Fold")
plt.ylabel("Recall (Severe = 0)")
```

```
plt.ylim(0, 1)
plt.xticks(folds)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```




Balanced Random Forest achieved higher Severe-class recall (0.583 ± 0.035) than Logistic Regression (0.558 ± 0.019), but with greater variance. Logistic Regression offers more stable performance across folds.

In the Figure, BRF shows stronger but less consistent recall, while LR is more stable.

## 5.3 Feature Importance

In [214…

```
from sklearn.inspection import permutation_importance

# Permutation importance for Balanced Random Forest
result_brf = permutation_importance(
    brf_best, X_val, y_val,
    n_repeats=10,
    random_state=42,
    scoring='f1_macro'
)

# Top features
importance_brf = pd.DataFrame({
    'feature': X_val.columns,
```

```
    'importance_mean': result_brf.importances_mean,
    'importance_std': result_brf.importances_std
}).sort_values(by='importance_mean', ascending=False)

importance_brf.head(10)
```

Out[214]:

| | feature | importance_mean | importance_std |
|---|---|---|---|
| **0** | day_of_week | 0.009322 | 0.003763 |
| **8** | month | 0.008665 | 0.005876 |
| **13** | region_number | 0.008116 | 0.004233 |
| **6** | age_of_driver | 0.007671 | 0.007864 |
| **7** | driver_imd_decile | 0.006433 | 0.004213 |
| **11** | light_condition_label | 0.005760 | 0.002843 |
| **4** | accident_year | 0.005459 | 0.005318 |
| **2** | road_type | 0.004419 | 0.005906 |
| **3** | urban_or_rural_area | 0.003549 | 0.003673 |
| **17** | propulsion_code_grouped | 0.002981 | 0.001709 |

In [215…

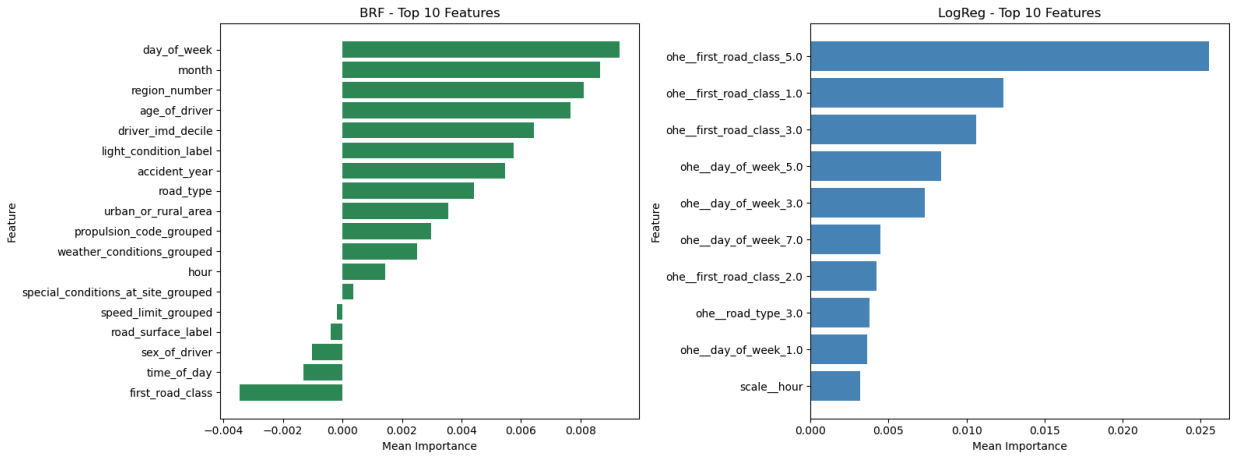
```
# Permutation Importance for Logistic Regression
result_lr = permutation_importance(
    lr_best, X_val, y_val,
    n_repeats=10, random_state=42, scoring='f1_macro'
)

# Get correct feature names from pipeline
lr_feature_names = lr_best.named_steps['pre'].get_feature_names_out()
n = len(result_lr.importances_mean)
lr_feature_names = lr_feature_names[:n]  # ensure alignment

importance_lr = pd.DataFrame({
    'feature': lr_feature_names,
    'importance_mean': result_lr.importances_mean,
    'importance_std': result_lr.importances_std
}).sort_values(by='importance_mean', ascending=False).head(10)

importance_lr.head(10)
```

Out[215]:

| | feature | importance_mean | importance_std |
|---|---|---|---|
| **13** | ohe__first_road_class_5.0 | 0.025551 | 0.007336 |
| **9** | ohe__first_road_class_1.0 | 0.012342 | 0.002861 |
| **11** | ohe__first_road_class_3.0 | 0.010617 | 0.004333 |
| **6** | ohe__day_of_week_5.0 | 0.008373 | 0.005167 |
| **4** | ohe__day_of_week_3.0 | 0.007303 | 0.006263 |
| **8** | ohe__day_of_week_7.0 | 0.004468 | 0.004192 |
| **10** | ohe__first_road_class_2.0 | 0.004217 | 0.002750 |
| **17** | ohe__road_type_3.0 | 0.003794 | 0.002406 |
| **2** | ohe__day_of_week_1.0 | 0.003654 | 0.006857 |
| **1** | scale__hour | 0.003192 | 0.002820 |

In [216…

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# BRF plot
axes[0].barh(importance_brf['feature'][::-1], importance_brf['importance_mean'][::-1],
axes[0].set_title("BRF - Top 10 Features")
axes[0].set_xlabel("Mean Importance")
axes[0].set_ylabel("Feature")

# LR plot
axes[1].barh(importance_lr['feature'][::-1], importance_lr['importance_mean'][::-1], c
axes[1].set_title("LogReg - Top 10 Features")
axes[1].set_xlabel("Mean Importance")
axes[1].set_ylabel("Feature")

plt.tight_layout()
plt.show()
```




**Interpretation:** Balanced Random Forest identifies key predictors like day_of_week, month, and region_number, suggesting that **time and location patterns strongly influence accident severity**. Logistic Regression emphasizes specific road classes and traffic timing, showing that **certain road types and days are consistently riskier**.

These insights can directly support:

- **Risk-based routing** (e.g., avoid risky regions at peak times).
- **Dynamic pricing models** based on time and location risk.
- **Driver training targeted by road type and schedule**.
- **Insurance cost reduction** by proactively identifying high-risk bookings.

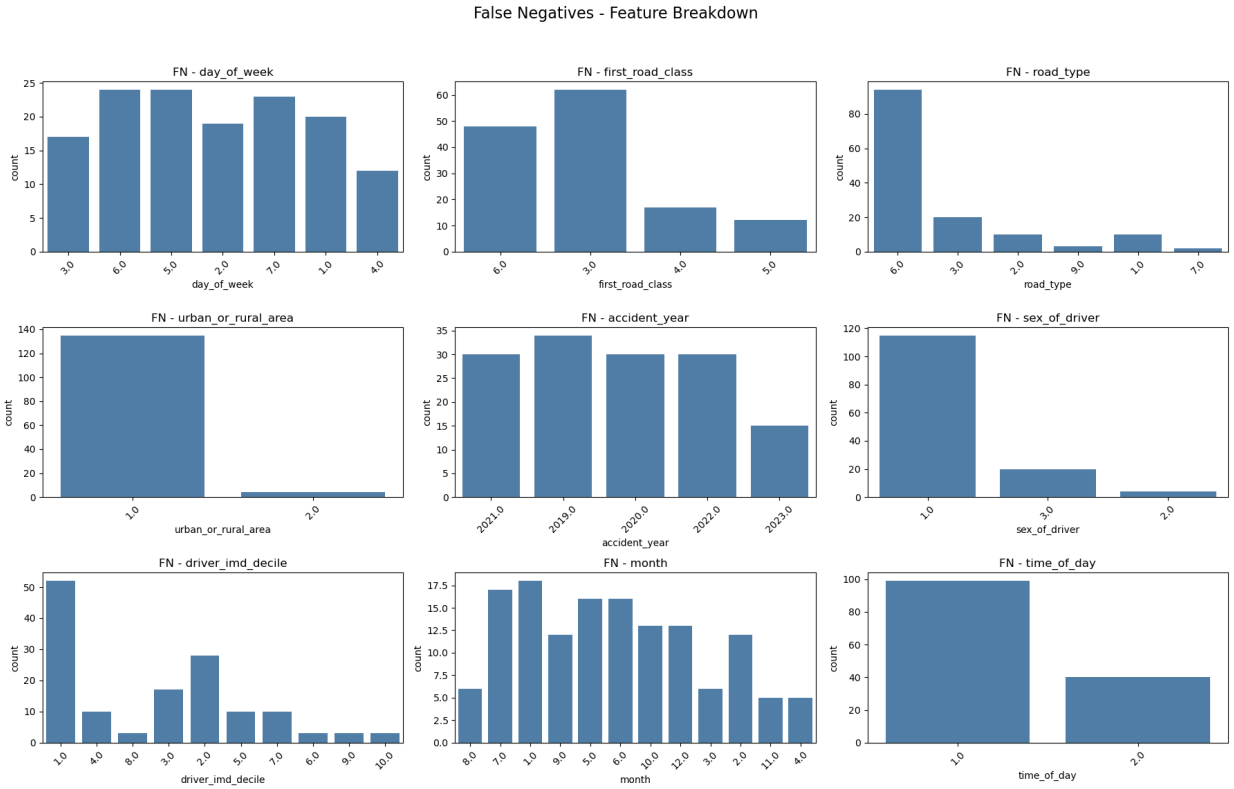
Together, the models help ride-hailing platforms take **proactive, data-driven actions before a trip even begins**.

## 5.4 Error Pattern Analysis

We analyzed false negatives and false positives to understand when and why the model fails, revealing patterns like urban daytime crashes or high-risk roads that may be misclassified. We focused on Logistic Regression since it showed more consistent generalization than Balanced Random Forest, making it more reliable for uncovering meaningful and actionable error patterns.

In [217…

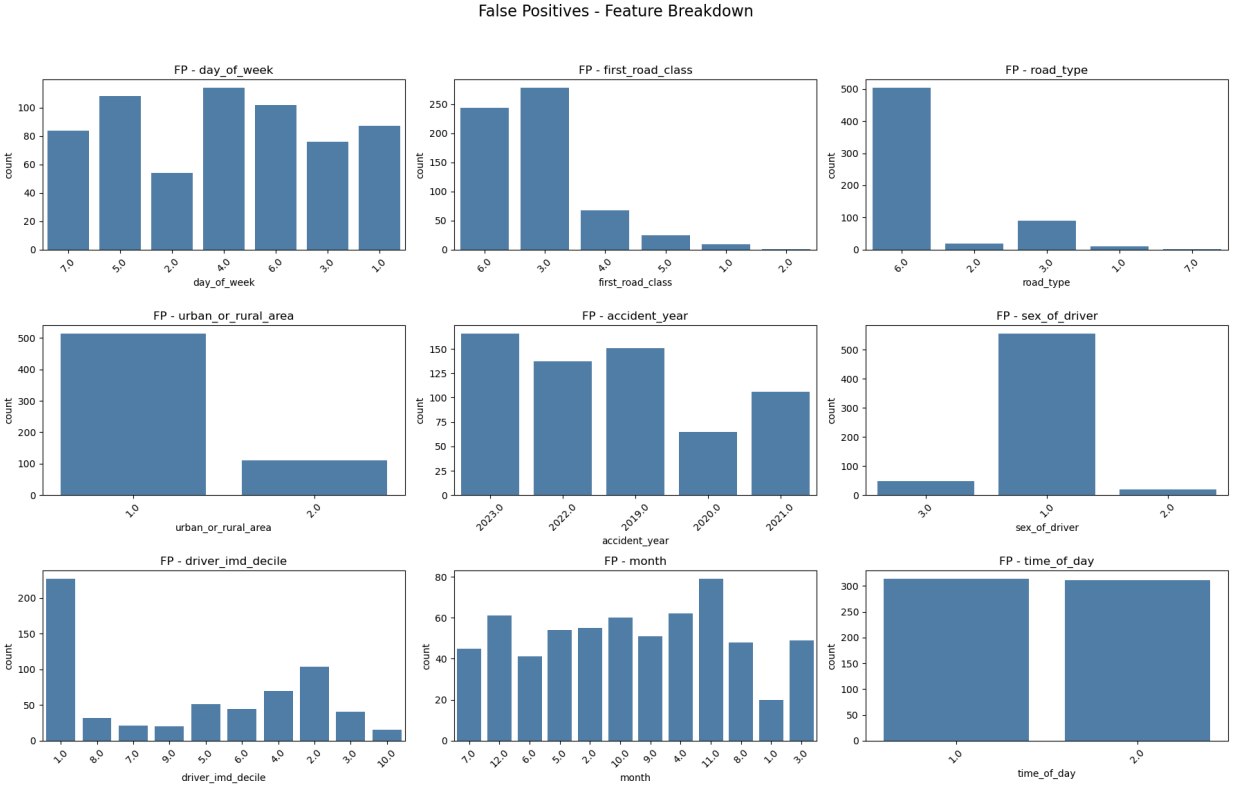
```
import matplotlib.pyplot as plt
import seaborn as sns

# Get all categorical columns
categorical_cols = df_val.select_dtypes(include='object').columns.tolist()

# FALSE NEGATIVES
fig, axes = plt.subplots(3, 3, figsize=(18, 12))
axes = axes.flatten()

for i, col in enumerate(categorical_cols):
    if i >= len(axes): break
    sns.countplot(x=fn[col], ax=axes[i], color='steelblue')
    axes[i].set_title(f"FN - {col}")
    axes[i].tick_params(axis='x', rotation=45)

# Remove extra axes
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle("False Negatives - Feature Breakdown", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```




False negatives were most common in urban areas, during daylight hours, and among male drivers from highly deprived backgrounds. These errors suggest the model may be underestimating risk in scenarios typically perceived as lower danger such as daytime city driving. Additionally, frequent misses on A-roads and minor roads imply the model lacks contextual nuance around high-traffic routes.

In [218…

```
# FALSE POSITIVES
fig, axes = plt.subplots(3, 3, figsize=(18, 12))
axes = axes.flatten()

for i, col in enumerate(categorical_cols):
    if i >= len(axes): break
    sns.countplot(x=fp[col], ax=axes[i], color='steelblue')
    axes[i].set_title(f"FP - {col}")
    axes[i].tick_params(axis='x', rotation=45)

# Remove extra axes
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle("False Positives - Feature Breakdown", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```




In contrast, false positives were also concentrated in urban settings, especially involving deprived drivers, and occurred year-round, peaking in late-year months. The model appears to over-respond to common indicators like road type, deprivation level, and time of year without enough situational context, leading to unnecessary severity flags. This reflects a cautious bias that may protect against severe-case misses but reduces model precision and could strain operational response systems.

# 6. Conclusion

The objective of this project was to predict the severity of potential taxi collisions using pre-trip data, enabling ride-hailing platforms to proactively manage risk, improve safety, and support smarter operational decisions. As severe incidents are rare but costly, the evaluation prioritized recall to minimize missed high-risk trips.

Logistic Regression was selected as the primary model due to its stable validation performance and high interpretability. Balanced Random Forest was retained as a secondary model, offering strong recall but with signs of overfitting.

## Recommendation

- Deploy the model at the time of booking to flag high-risk trips early and allow for proactive interventions (e.g., alternate routing or driver alerts).
- Regularly retrain the model to reflect changes in traffic patterns, seasonality, and infrastructure.

- Apply threshold tuning or cost-sensitive classification to reduce false positives while maintaining high recall.
- Incorporate additional real-time features (e.g., live traffic, weather, or driver indicators) if available.

## Limitations

- The model uses only structured, pre-trip data and lacks real-time inputs such as GPS speed, traffic congestion, or environmental conditions.
- Low precision may trigger excessive alerts, requiring well-defined response protocols.
- Model generalisation may drift over time; ongoing retraining is necessary to sustain performance.

## Deployment and Next Steps

- Integrate the model into the booking or dispatch system to flag risky journeys before they begin.
- Expose model outputs via a dashboard or API for real-time scoring and monitoring.
- Continuously evaluate model predictions and gather driver/operator feedback to refine thresholds and build trust in system decisions.